DOCKER

NAME: S.GAYATHERI COURSE CODE: CSA1013

REG NO : 192521306 COURSE NAME: SOFTWARE ENGINEERING

EXP:01-**Create a Static Website and Containerize, Build & Serve it using Docker**

AIM

To create a simple static website using HTML, containerize the website using Docker and Nginx, build a Docker image, run the website inside a Docker container, and access it through a web browser.

WORKFLOW

Step 1: Create the static website

```
<!DOCTYPE html>
<html>
<head>
    <title>My Docker Website</title>
</head>
<body>
    <h1>Welcome to My Docker Website</h1>
    <p>This website is running inside a Docker container.</p>
    <p>It is served using Nginx.</p>
</body>
</html>
```

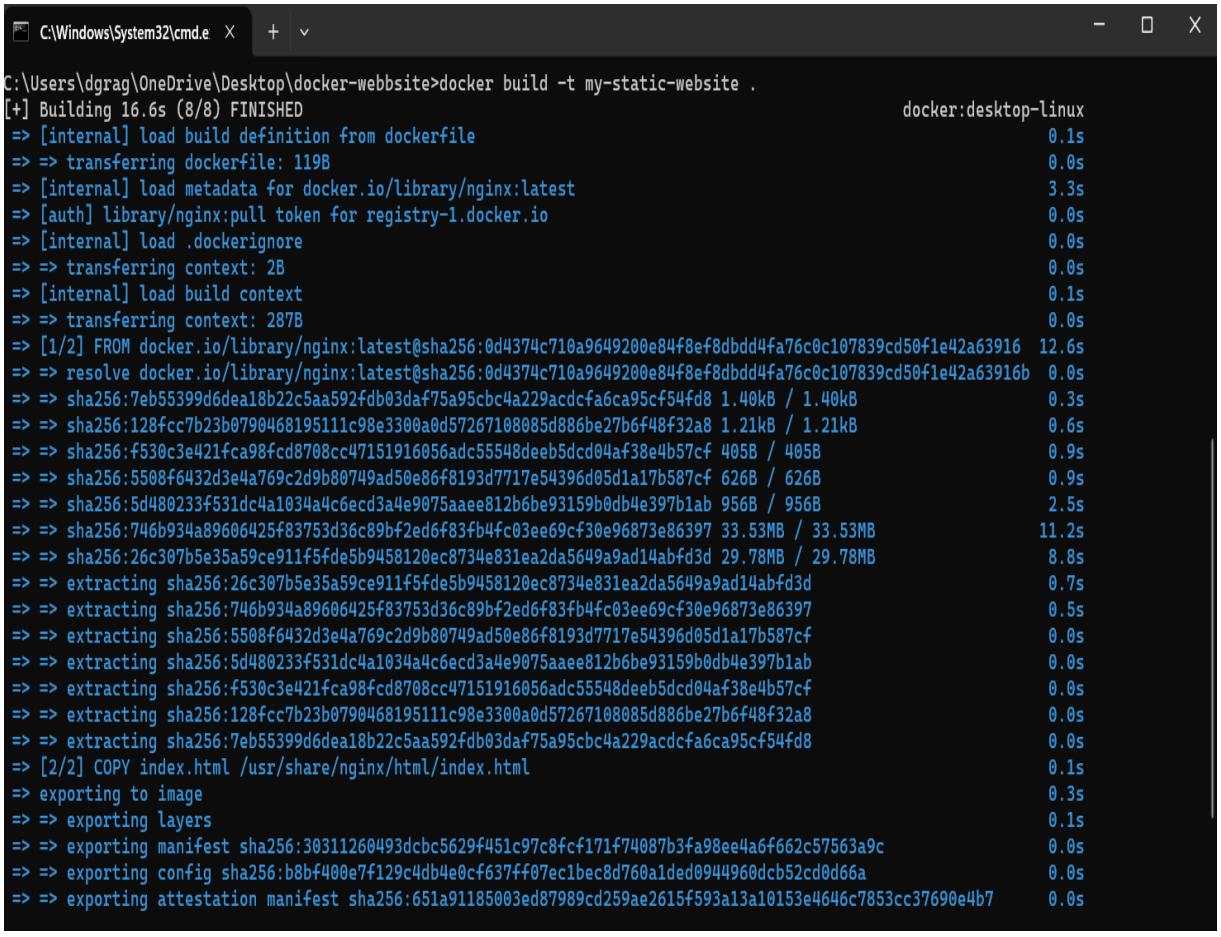
Step 2: Create the Dockerfile

```
FROM nginx:latest

COPY index.html /usr/share/nginx/html/index.html

EXPOSE 80
```

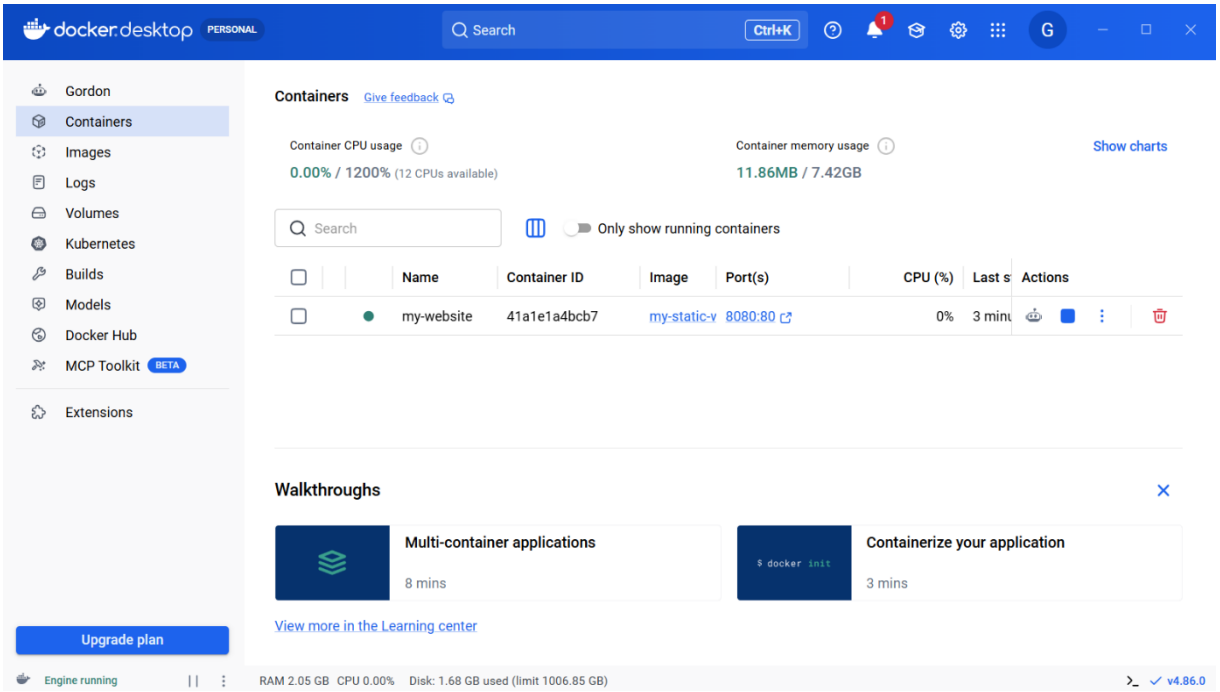
Step 3: Verify the project files

```
C:\Users\dgrag\OneDrive\Desktop\docker-webbsite>dir
 Volume in drive C has no label.
 Volume Serial Number is E4CA-8963

 Directory of C:\Users\dgrag\OneDrive\Desktop\docker-webbsite

23-08-2026  16:20    <DIR>          .
23-08-2026  15:39    <DIR>          ..
23-08-2026  15:22                82 dockerfile
23-08-2026  15:21               248 index.html
               2 File(s)            330 bytes
               2 Dir(s)  364,538,810,368 bytes free
```

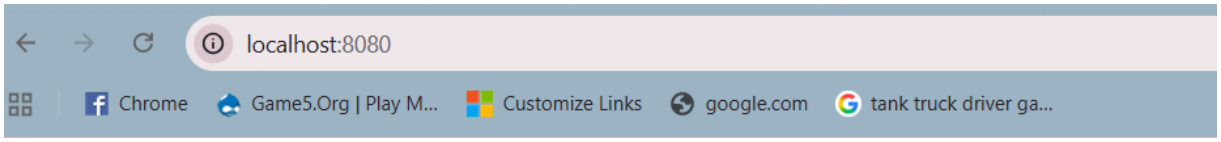
Step 4: Build the Docker image



Step 5: Run the Docker container



Step 6: Access the website



Welcome to My Docker Website

This website is running inside a Docker container.

It is served using Nginx.

RESULT

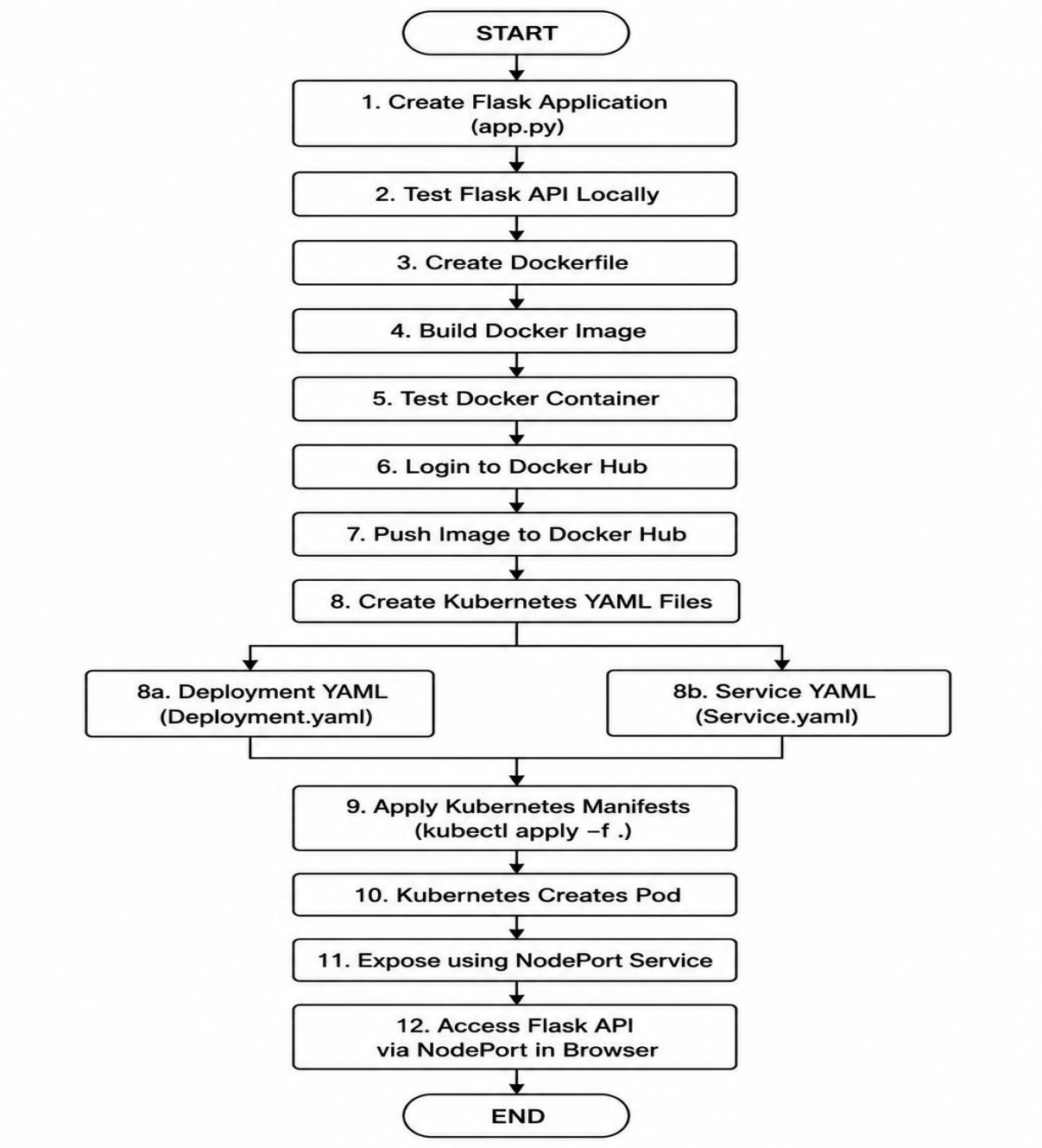
Thus, a static website was successfully created using HTML, containerized using Docker with Nginx, built into a Docker image, and run as a Docker container. The website was successfully served and accessed through the web browser using http://localhost:8080.

EXP:02- **Containerization and Kubernetes Deployment of a Python Flask API Using Docker**

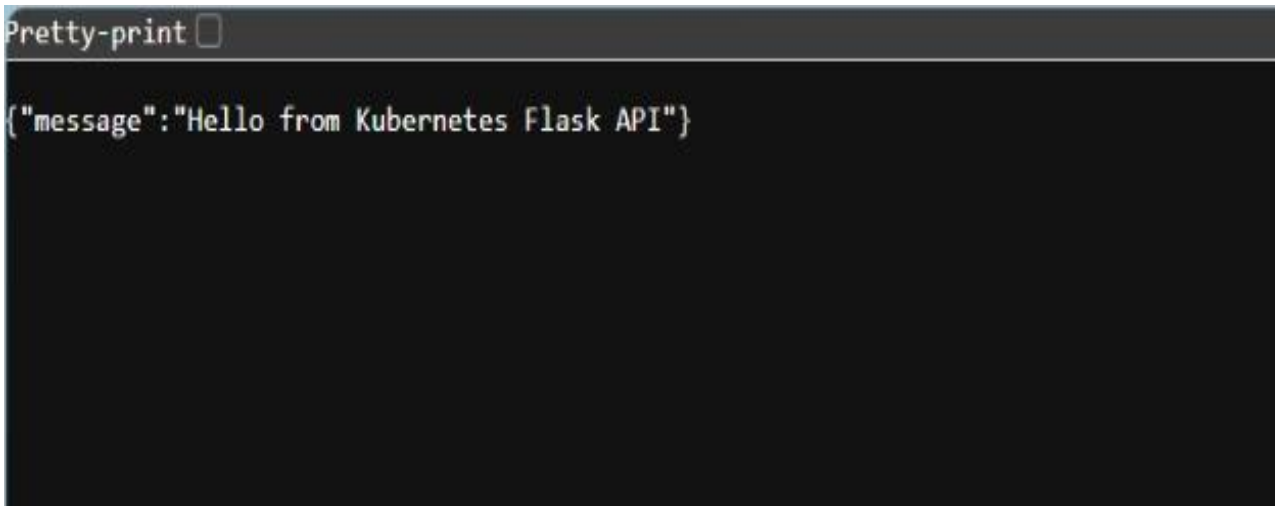
AIM

To develop a simple **Python Flask REST API**, containerize it using **Docker**, build and push the Docker image to **Docker Hub**, and deploy the application on **Kubernetes** using a **Deployment and NodePort Service**.

FLOWCHART



Successful Kubernetes Flask API Response



RESULT:

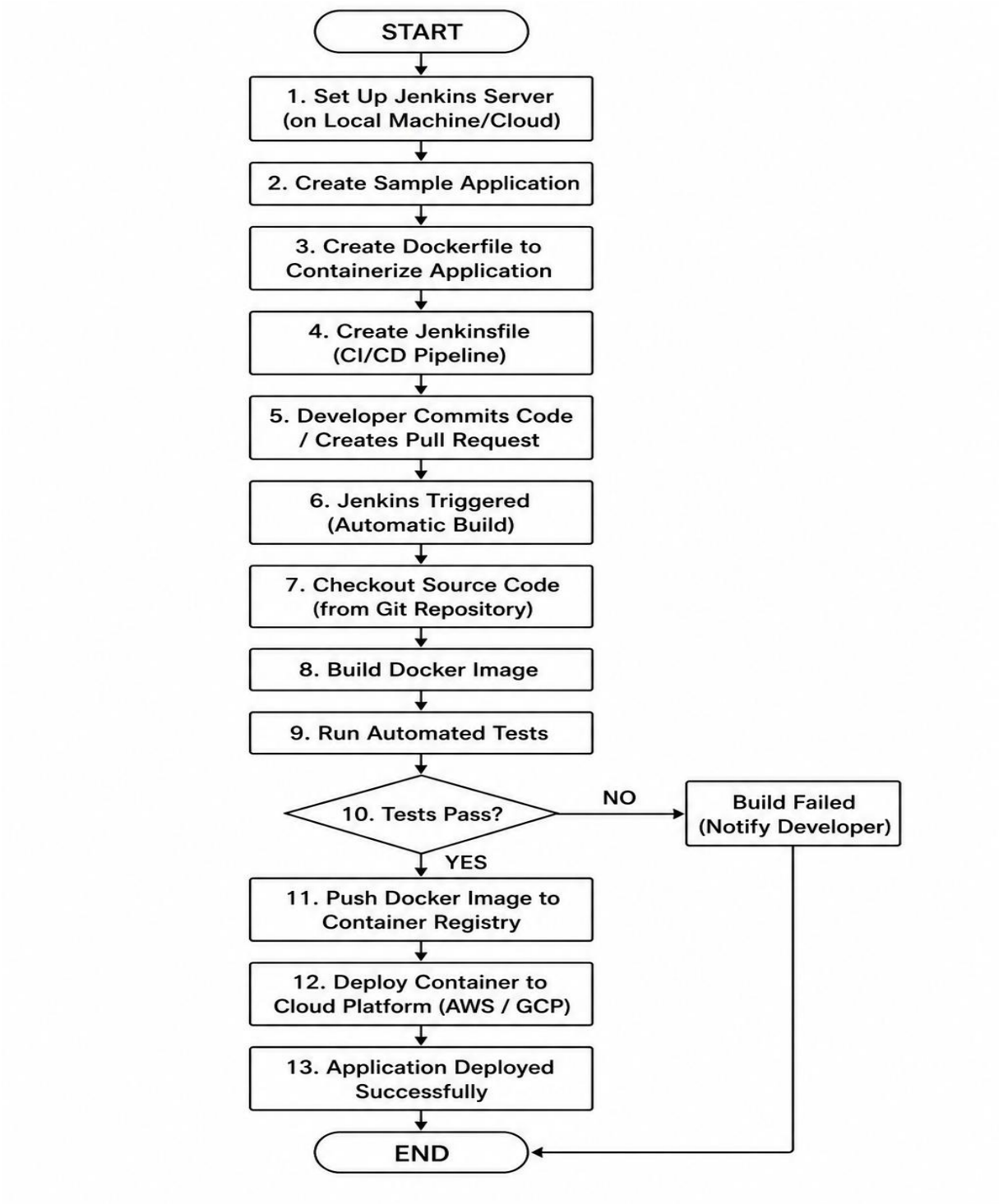
The Flask API was successfully created, containerized using Docker, built and pushed to Docker Hub, and deployed on Kubernetes using Deployment and NodePort Service manifests. The API was successfully accessed through the Kubernetes NodePort service.

EXP 3 - CI/CD Pipeline for Containerized Application Using Jenkins and Docker

Aim

To set up a CI/CD pipeline using Jenkins that automatically builds, tests, and deploys a containerized application whenever new code is committed or a pull request is created.

FLOWCHART



CI/CD Pipeline Application Running on Localhost

Result

The **Jenkins CI/CD pipeline was successfully configured** to automate the application lifecycle. The pipeline automatically retrieves the latest code, builds a Docker image, executes tests, and deploys the containerized application to a cloud platform. Jenkins can also be configured to trigger the pipeline automatically whenever code is committed or a pull request is created.

EXP 4 - Continuous Deployment Using GitHub Actions and Docker

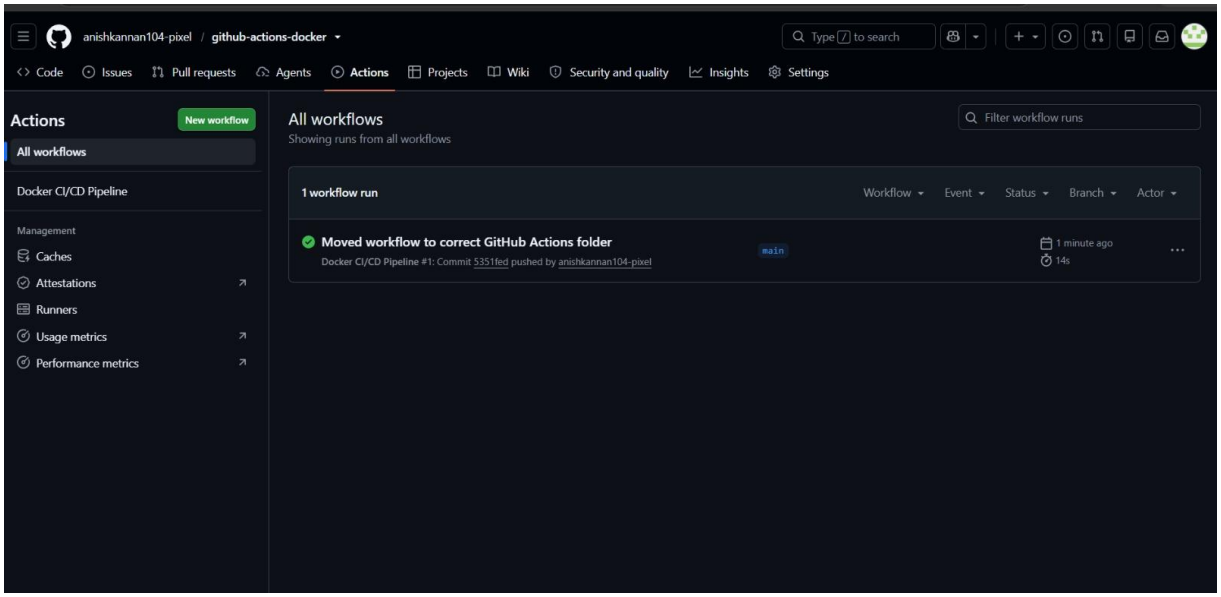
Aim

To implement Continuous Deployment (CD) using GitHub Actions by automatically building and pushing a Docker image and deploying the Dockerized application to a cloud platform whenever new code changes are pushed to the GitHub repository.

Procedure

1. Create a GitHub repository.
2. Create the sample application.
3. Create a Dockerfile.
4. Build and test the Docker image locally.
5. Push the application to GitHub.
6. Create .github/workflows/deploy.yml.
7. Add Docker Hub/cloud credentials as GitHub Secrets.
8. Push the workflow file to the main branch.
9. GitHub Actions automatically starts the workflow.
10. Docker image is built.
11. Docker image is pushed to Docker Hub/GitHub Container Registry.
12. The cloud deployment is triggered.
13. Make another code change and push it to GitHub.
14. Verify the new GitHub Actions run.
15. Verify that the updated application is running on the cloud platform.

Successful GitHub Actions Docker CI/CD Pipeline Run



Result

The GitHub Actions CI/CD pipeline executed successfully, automatically building and deploying the Dockerized application.

EXP 5 - Containerize a Python Flask To-Do Application Using Docker

Aim

To develop a simple Python Flask To-Do application, containerize it using Docker, test it locally inside a Docker container, and push the Docker image to Docker Hub.

Requirements

- Python
- Flask
- Docker Desktop
- Docker Hub account
- Web browser

PROGRAM

```
from flask import Flask, request, jsonify
app = Flask(__name__)
todos = []
@app.route("/")
def home():
    return "Flask To-Do Application is Running!"
@app.route("/todos", methods=["GET"])
def get_todos():
    return jsonify(todos)
@app.route("/todos", methods=["POST"])
def add_todo():
    data = request.get_json()
    todo = {
        "id": len(todos) + 1,
        "task": data["task"]
    }
    todos.append(todo)
    return jsonify(todo), 201
@app.route("/todos/<int:todo_id>", methods=["DELETE"])
def delete_todo(todo_id):
    global todos
    todos = [todo for todo in todos if todo["id"] != todo_id]
    return jsonify({"message": "Todo deleted successfully"})
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Flask To-Do List Application Running Successfully

Result

The Flask To-Do application was successfully containerized using Docker, tested locally, and the Docker image was pushed to Docker Hub.

EXP 6 - Push and Pull Docker Images Using Docker Hub

Aim

To create a Docker image for a static HTML website, push the image to Docker Hub, pull it onto another machine, and run the website using Docker**.**

Requirements

- Docker Desktop
- Docker Hub account
- Static HTML file
- Internet connection
- Two machines, or Docker environments

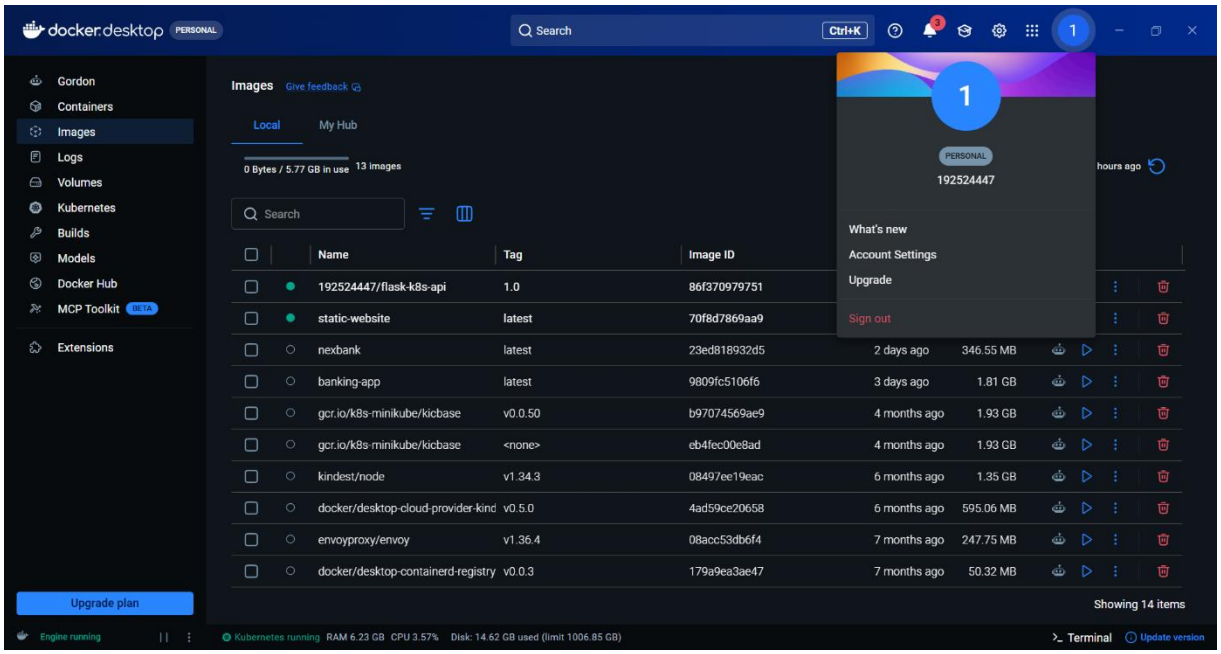
1. Create index.html

```
<!DOCTYPE html>
<html>
<head>
   <title>Docker Website</title>
</head>
<body>
   <h1>Welcome to My Docker Website</h1>
   <p>This website is running from a Docker container.</p>
</body>
</html>
```

2. Create Dockerfile

```
FROM nginx:latest

COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

Docker Desktop Showing Static Website Docker Image



Result

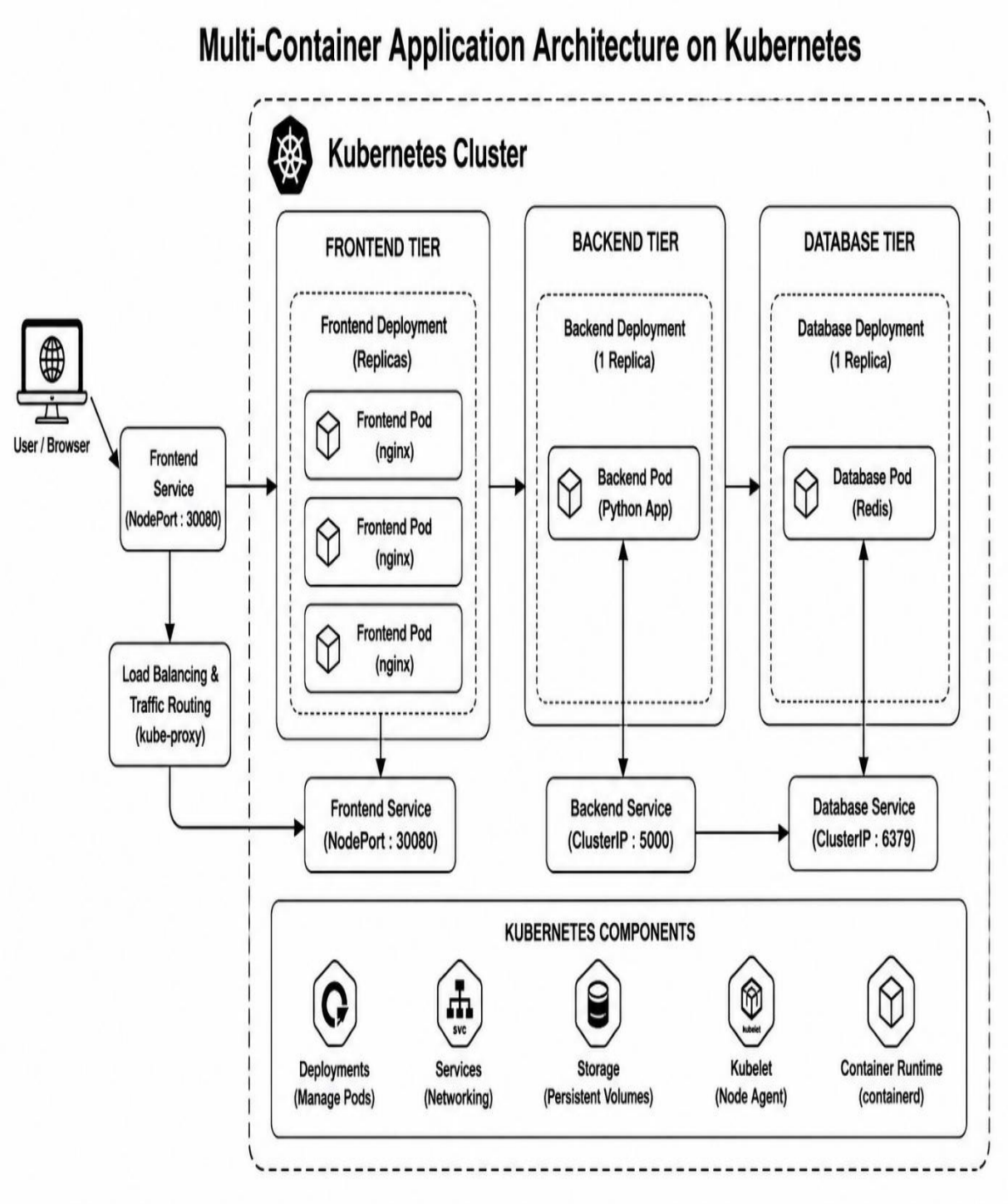
The static website Docker image was successfully built, tagged, pushed to Docker Hub, pulled onto another machine, and executed successfully using Docker.

EXP 7 - Deploy a Multi-Container Application Using Kubernetes

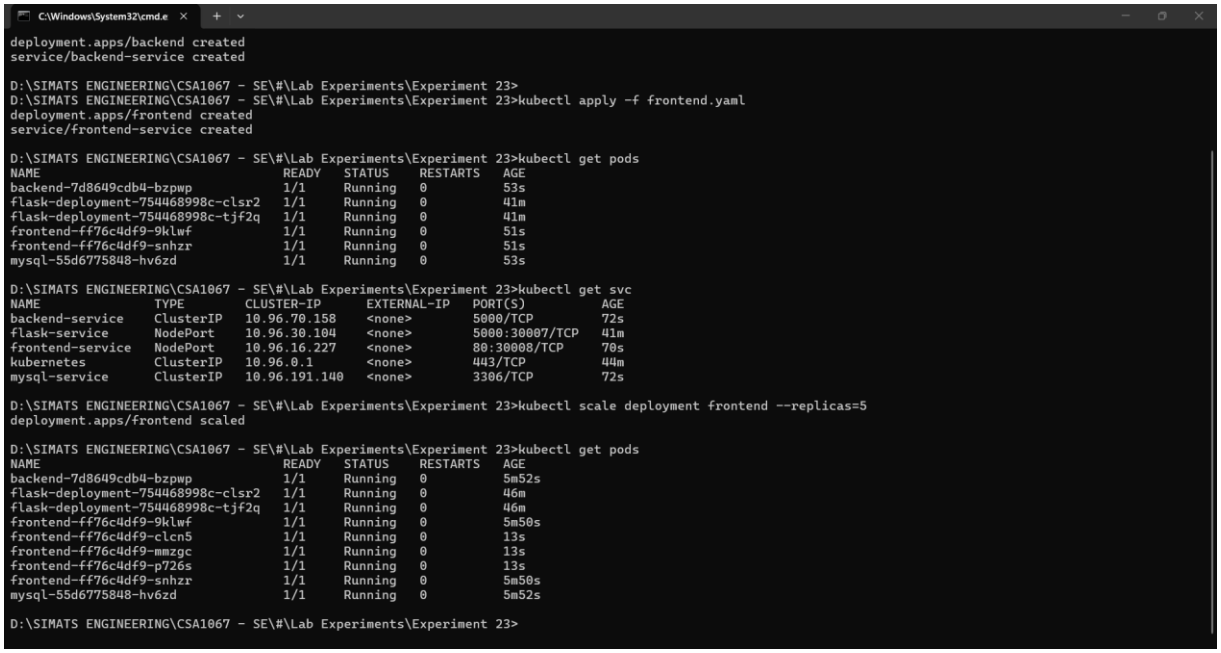
Aim

To deploy a multi-container application consisting of frontend, backend, and database components using Kubernetes, expose the services, monitor their status, and scale the frontend to handle increased load.

Architecture



Kubernetes Multi-Container Application Deployment and Frontend Scaling



Result

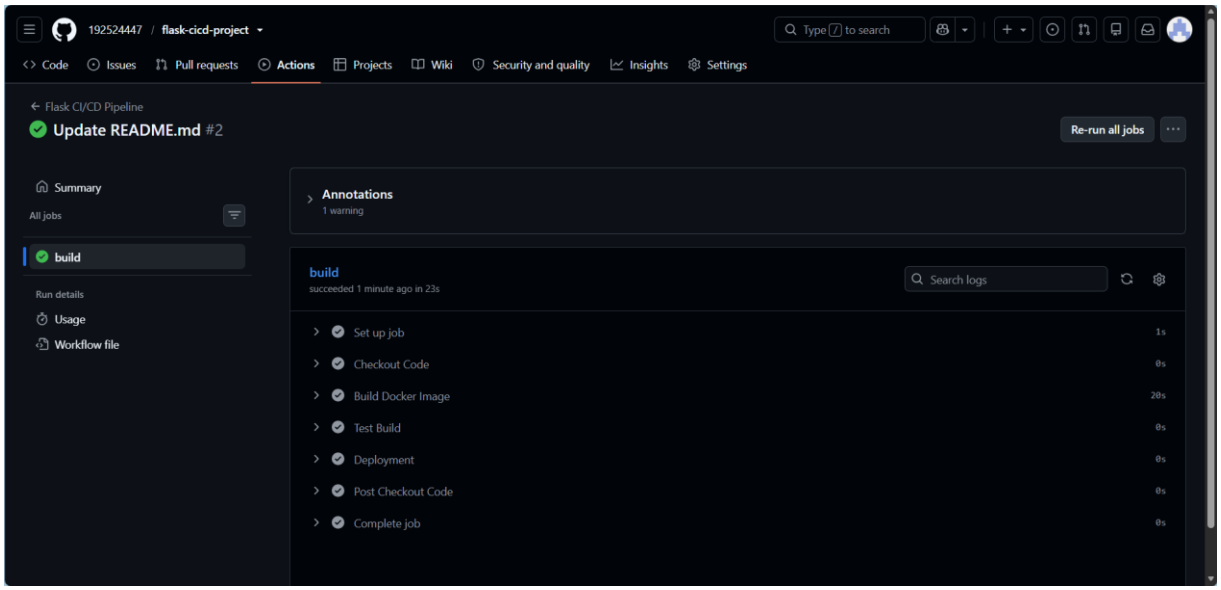
The multi-container application was successfully deployed using Kubernetes with frontend, backend, and database components. The pods and services were monitored successfully, and the frontend was scaled from 2 to 3 replicas to handle increased load.

EXP 8 - Create a CI/CD Pipeline Using GitHub Actions

Aim

To create a CI/CD pipeline using GitHub Actions for a containerized Flask application that automatically tests the code, builds and pushes the Docker image to Docker Hub, and deploys the application to a cloud platform.

Successful GitHub Actions CI/CD Pipeline for Flask Application



Result

The GitHub Actions CI/CD pipeline was successfully created to automate testing, Docker image building, and pushing to Docker Hub, followed by cloud deployment. The final deployment URL can be submitted along with the workflow file.